most-exchange FIX adapter

Operator's Manual

Installing, configuring and running the FIX 4.4 adapter in front of most-exchange: its two FIX engines, the order store and archive that make delivery guaranteed, the development stack beside most-exchange's own, and what to do when something goes wrong.

Revision 2026-10-31

Applies to the current implementation. The control API, metrics and session schedule are designed, not built.

Sections marked NOT IMPLEMENTED describe behaviour that is designed but not built.

© 2026 P.M.Vrolijk. Licensed under CC BY 4.0: https://creativecommons.org/licenses/by/4.0/

Contents

1 **Introduction**

1.1 What this adapter is

1.2 The process and its parts

1.3 Two boundaries

1.4 The two kinds of configuration

1.5 Prices, quantities and identifiers

1.6 Sessions and participants

1.7 What is not built

1.8 How to read this manual

2 **Development deployment and operation**

2.1 Prerequisites

2.2 Build

2.3 The dev stack beside most-exchange’s

2.4 Verifying the stack came up

2.5 The FIX client

2.6 Driving a trade

2.7 Watching the book

2.8 Stopping, restarting, crashing

2.9 What is development-only

2.10 Running without Docker

2.11 Automated checks

2.12 Shutting down

3 **Production deployment**

3.1 The rule that shapes everything

3.2 Topology

3.3 Machine preparation

3.4 Threads and cores

3.5 Aeron

3.6 Gateways and failover

3.7 Market data feeds

3.8 Process management

3.9 Bringing an adapter up

3.10 Blocking gaps

4 **Configuration in full**

4.1 How the adapter is configured

4.2 Order entry’s engine

4.3 Order entry’s exchange links

4.4 The trading calendar
4.5 Market data
4.6 Threads and the trace ring
4.7 The sessions file
4.8 JVM flags
4.9 What the exchange must agree to
4.10 A complete node file

5 **Daily operations, order flow and monitoring**
5.1 The trading day
5.2 Sessions
5.3 The order path
5.4 Delivery
5.5 An order whose fate is unknown
5.6 Restarts and crashes
5.7 Market data operations
5.8 Measuring
5.9 Monitoring

6 **Troubleshooting and maintenance**
6.1 First moves
6.2 Startup failures
6.3 Logon problems
6.4 Order flow problems
6.5 Aeron and Docker problems
6.6 After a crash
6.7 Maintenance procedures
6.8 Reading the logs
6.9 Consolidated list of what is not implemented

7 **Index**
7.1 Terms and concepts
7.2 Commands
7.3 Configuration keys and environment variables
7.4 Further reading

1. Introduction

1.1 What this adapter is

The most-exchange FIX adapter puts a FIX 4.4 interface in front of most-exchange. A participant's FIX engine connects to it over TCP; the adapter speaks to the exchange's order entry gateways and market data feeds over Aeron, in the exchange's binary SBE protocol, through most-exchange's own `client` SDK. It is one Java process, built on Artio, the FIX engine of the Aeron family.

It is a protocol adapter and nothing more. It adds no order types, no risk checks and no matching: an order is accepted at the adapter only if the exchange can carry it, and every exchange rule is the exchange's (most-exchange's `docs/Adapters.md`, which this adapter implements).

What it does add is **state**. The exchange delivers each execution report once, numbered, and the adapter promises the FIX client that no report is lost and no order's fate is misreported, across its own crashes, the client's disconnections and a gateway's failover. Most of this manual's operational advice follows from that promise.

> **PARTICIPANT**
> The exchange's trading identity: a number in the exchange's participant registry. Every order carries one. In the adapter, a FIX order-entry session acts for exactly one participant, set in its configuration; nothing on a FIX message can name another.

> **ORDER ENTRY, MARKET DATA**
> The adapter's two **roles**. An order-entry session sends orders and cancels and receives execution reports. A market-data session asks for the security list and subscribes to books. A session has one role, fixed in the configuration, and logs on at that role's own port.

1.2 The process and its parts

One adapter process runs two FIX engines, each with its own port, embedded media driver and archive:

Part	What it does
Order entry's Artio engine	Accepts the order-entry sessions on `fix.port`. Records every FIX message in and out in an **archive fsynced to disk**
Order entry's library thread	Decodes and validates 35=D, F, H and AF; writes the **order store**; sends to the exchange once the archive has the request; turns the exchange's reports into 35=8 and 35=9
The order store	RocksDB, in `fix.storeDir`: one row per order and per cancel, and how far each participant's reports have been durably delivered
Market data's Artio engine	Accepts the market-data sessions on `fix.md.port`, with an archive that is not synced and is emptied at every start
Market data's library thread	Answers 35=x and 35=V from the exchange's directory and feeds
The embedded media drivers	One per engine. Order entry's also carries the exchange links: the gateways and the directory

Order entry and market data are separate engines on purpose. Order entry's archive is fsynced and every order waits for it (section 5.3); market data's messages on the same archive slowed order entry measurably, so they have their own (Design §4).

> **ARCHIVE**
> Aeron Archive's recording of every message an Artio engine sends and receives, in `fix.archiveDir`. Order entry's is synced at `fileSyncLevel` 1: what it has recorded survives a host crash. It is what FIX resends come from, and what the adapter waits on before an order goes to the exchange and before a report is counted as delivered.

1.3 Two boundaries

The adapter sits between two networks, and is the only thing that should touch both:

- **The FIX side:** plain TCP, FIX 4.4, one port for order entry and one for market data. Artio does not encrypt; TLS, where needed, is terminated in front of the adapter (section 3.2).
- **The exchange side:** Aeron UDP to the exchange's gateway endpoints, its directory broadcast and its market data feeds. The adapter learns the gateways from the directory; it is never configured with the exchange's cluster.

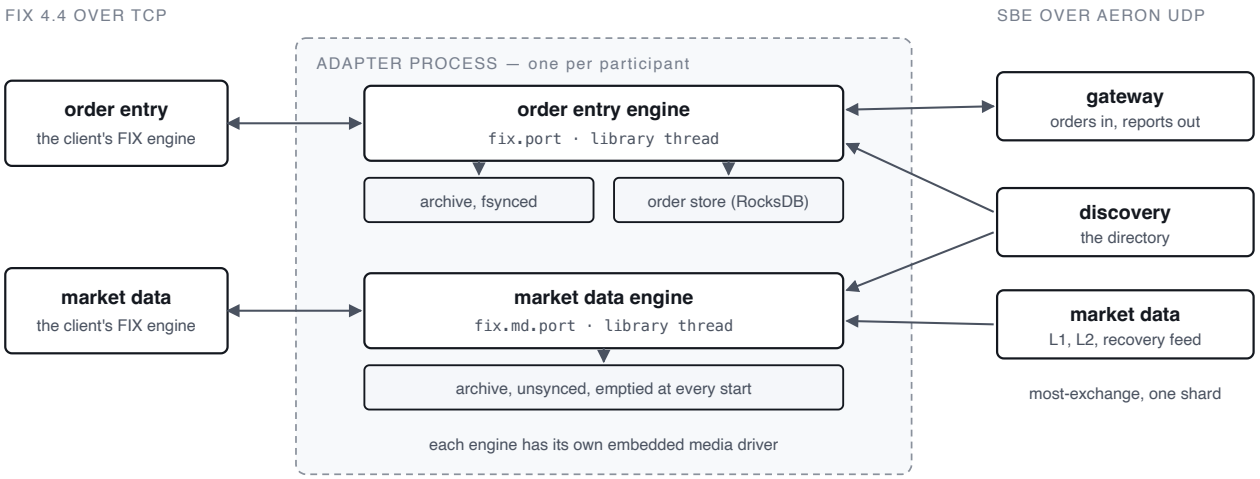


Figure 1.1 The adapter between FIX clients and the exchange

1.4 The two kinds of configuration

The adapter reads two files, both Java properties:

- **The node file**, `fix.*` keys: ports, directories, the exchange's channels, the calendar, the threads. Given as the only argument: `adapter /config/adapter.properties`. Any `fix.*` JVM system property overrides the file's key (section 4.1).
- **The sessions file**, named by `fix.sessionsFile`: one block per FIX session, with its CompIDs, username, the SHA-256 of its password, its role and, for order entry, its participant (section 4.7).

The adapter checks both completely at start and refuses to start on anything missing or malformed, before it accepts a single Logon.

The exchange has configuration of its own that must agree: every participant the adapter's sessions act for must be one the gateway it connects through may speak for (section 4.9).

1.5 Prices, quantities and identifiers

Prices and quantities are decimals on FIX and fixed-point integers at the exchange, with eight implied decimals: a price of `100.25` is `10025000000` at the exchange. The adapter converts exactly. A value with more than eight decimals is refused, never rounded. Tools that speak to the exchange directly, most-exchange's `most` CLI among them, take the raw integer: `--qty 400000000` is four shares.

Identifiers a client sees:

Identifier	Tag	What it is
ClOrdID	11	The client's, unique per session for the trading day and while the order is open
OrderID	37	The exchange's order id; `NONE` until the order is acknowledged
ExecID	17	The exchange's report sequence number, in decimal, for a report from the exchange; `A<index>.<MsgSeqNum>` for a message the adapter answered itself

The exchange's own `clOrdId` is not the client's: the adapter derives it from the FIX session, `(sequenceIndex << 32) | MsgSeqNum` of the 35=D or 35=F. That is why the session's sequence numbers matter to the store, and why wiping the adapter's state while the exchange keeps the participant's orders is unsafe (section 6.7).

1.6 Sessions and participants

- **One order-entry session, one participant.** Two order-entry sessions may not share a participant: reports route back one to one.
- **The adapter is the acceptor.** Its CompID is the session's `senderCompId`; the client's is `targetCompId`.
- **Order-entry sessions are persistent.** Their sequence numbers survive Logouts and adapter restarts, and reset only when the client logs on with ResetSeqNumFlag (141=Y). A report for a client that is logged out is written to its session offline, and resent at its next Logon.
- **Market-data sessions are transient.** They start at MsgSeqNum 1 at every Logon, and keep nothing between Logons.
- **Deployment:** one adapter per participant, with that participant's order-entry session and a market-data session beside it.

1.7 What is not built

Designed in `docs/Design.md`, not built yet:

- the control API (`/status`, `/start`, `/stop`, session resets, a trading date override) and `/metrics` (Design §9.1–§9.2);
- a session schedule: a configured session may log on at any time (Design §8.2);
- several order-entry sessions for one participant (Design §8.27).

The manual marks each in a *todo* callout where an operator would otherwise expect it to work; the consolidated list is in 6.9.

1.8 How to read this manual

Section 2 brings the adapter up beside most-exchange's development stack and is the fastest way to learn the vocabulary. Section 3 is production. Section 4 is the complete configuration reference, to be consulted rather than read. Section 5 is how the adapter behaves day to day: sessions, the order path, delivery, restarts. Section 6 is diagnosis and maintenance. Section 7 indexes terms, commands and every configuration key.

Command transcripts are real output from a running system, not illustrations.

The participant's side, the FIX messages and tags, is the companion document, the **Rules of Engagement** (`docs/RulesOfEngagement.pdf`). In the repository: `docs/Design.md` is the authoritative specification, `docs/Rationale.md` says why each rule exists, `deploy/README.md` covers the Docker setup, and most-exchange's `docs/OperatorManual.pdf` covers the exchange.

2. Development deployment and operation

This section brings participant 7's adapter up beside most-exchange's development stack, on one machine, in Docker. Everything here is development-only in the ways 2.9 lists; production is section 3.

2.1 Prerequisites

JDK 21	Builds the adapter and the `fix` client (`./gradlew`)
Docker with Compose	Docker Desktop on a Mac. Give its VM room: the exchange's stack and the adapter map about 3 GB of tmpfs between them (6.5)
A GitHub token	`read:packages`, for most-exchange's SDK: `githubUser` and `githubPackagesToken` in `~/.gradle/gradle.properties`, never in the repository or the shell
most-exchange	Checked out beside this repository as `../most-exchange`, its own stack built (`./gradlew installDist` there)

2.2 Build

```
./gradlew build                                  # compiles, runs every module's tests
./gradlew :adapter:installDist :tools:installDist # bin/adapter and bin/fix
```

`installDist` gives `adapter/build/install/adapter/bin/adapter`, the process, and `tools/build/install/fix/bin/fix`, the FIX client used throughout this manual. The SDK comes from most-exchange's GitHub Packages; `-PmostExchange.local=true` builds against `../most-exchange` instead, for changing the SDK and the adapter together.

2.3 The dev stack beside most-exchange's

The adapter is a compose project of its own, `most-fix`, in `deploy/`. It does not change most-exchange's stack; it joins two of its networks:

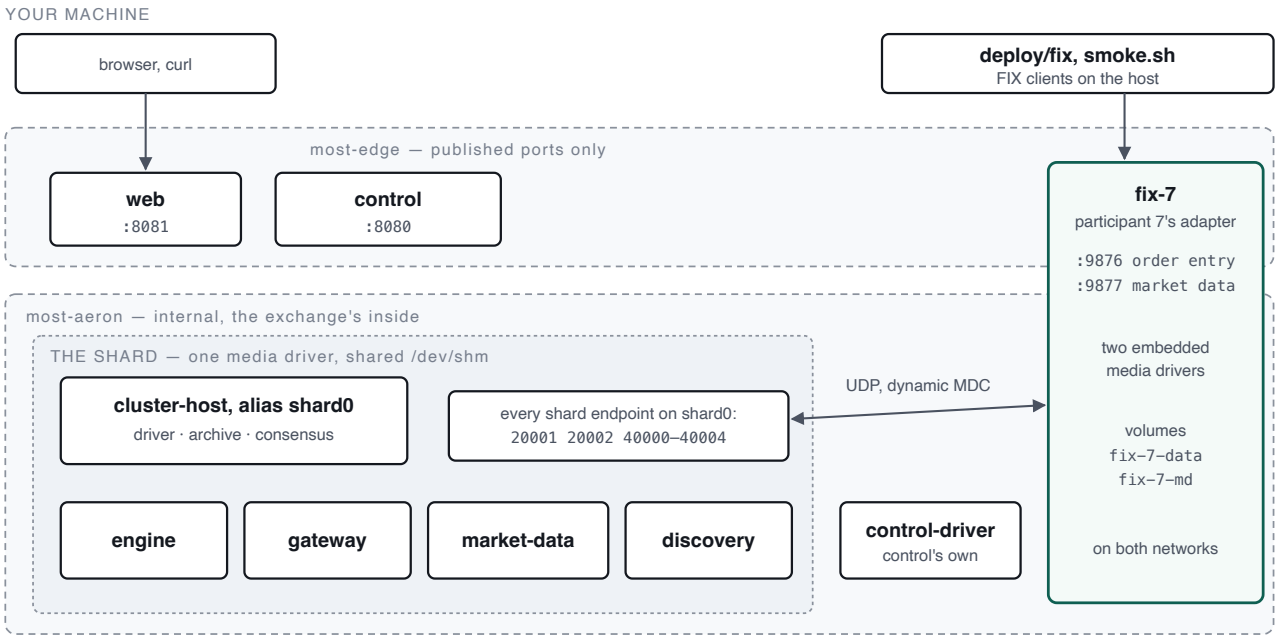


Figure 2.1 The adapter joins most-exchange's dev stack

- `most-aeron`, the exchange's inside, `internal: true`. The adapter reaches the gateway, the directory and the market data feeds over UDP here.
- `most-edge`, the only network with published ports. The adapter's FIX ports are published on it: 9876 for order entry and 9877 for market data.

The adapter is the one container on both, which is the place most-exchange's compose file leaves for an adapter. It brings its own media drivers, embedded; it shares no Aeron directory with any other container. Every exchange endpoint is on `shard0`, the network alias of most-exchange's `cluster-host`, whose media driver binds all of the shard's endpoints, and it subscribes by dynamic MDC because a Docker bridge does not route multicast.

```
(cd ../most-exchange/deploy && docker compose up -d)   # the exchange first: its networks
docker compose -f deploy/docker-compose.yml up -d --build
```

What the container is

Image	`deploy/Dockerfile`: the host's `adapter/build/install/adapter` on `eclipse-temurin:21-jre`, user 10001. No Gradle inside Docker
Configuration	`deploy/config/adapter.properties` and `sessions.properties`, mounted read-only at `/config`
Volumes	`fix-7-data` at `/data`: order entry's archive, Artio's logs, the store. `fix-7-md` at `/data-md`: market data's engine, emptied at every start
`/dev/shm`	1 GB, for the two embedded drivers
Stop	SIGTERM, 30 s grace: the clean stop (5.6)
Restart	`unless-stopped`: after a crash it retries until the archive's mark file has aged out (6.6)
Health	The order-entry port accepts a connection. The acceptor binds last, so an open port is a started adapter

The sessions, with development secrets committed as most-exchange's are:

Session	Port	CompIDs (client → adapter)	Username / password
Order entry, participant 7	9876	CLIENT7 → MOST	`client7` / `client7-secret`
Market data	9877	FEED7 → MOST	`feed7` / `feed7-secret`

2.4 Verifying the stack came up

```
$ docker logs most-fix-fix-7-1 2>&1 | grep -v SVE
adapter: started, listening on 0.0.0.0:9876 for 1 order-entry sessions, 0.0.0.0:9877 for 1 market-data; trading date 20261031, idle backoff
order entry: shard 0 through aeron:udp?endpoint=shard0:20001:20
order entry: shard 0 ready
```

Three lines, in this order:

1. `adapter: started` names both ports, the session counts, the trading date the calendar gives day orders, and the idle strategy. The ports are open from this line on.
2. `shard 0 through …` : the directory has arrived, and the adapter has a session with the gateway it names, as `channel:streamId` .
3. `shard 0 ready` : that session's opening mass status has finished. Orders for shard 0's securities go to the exchange from here; before it they are refused "exchange unavailable".

`started` without `shard 0 through` means no directory: see 6.2.

2.5 The FIX client

`deploy/fix` runs the host's `fix` client with participant 7's connection details filled in. Order commands log on as CLIENT7 at 9876; `securities` and `md` log on as FEED7 at 9877. The client keeps its MsgSeqNums in `build/deploy/clients` , so each command is a Logon, the request, and a Logout, in sequence with the last.

Command	Sends
`send --symbol S --side buy|sell --price P --qty Q --clordid ID [--tif day|gtc|ioc|fok]`	35=D
`cancel --clordid ID --orig ID --symbol S --side buy|sell --qty Q`	35=F
`status --clordid ID --symbol S --side buy|sell`	35=H
`mass-status [--side buy|sell]`	35=AF, all orders
`listen [--follow SECONDS]`	Only a Logon: prints what is owed
`securities`	35=x
`md --symbol S[,S…] [--depth 0|1]`	35=V, snapshot plus updates

Every application message received is printed on one line. `--follow` (default 3 s) is how long the client stays logged on after its request.

2.6 Driving a trade

A sell from participant 7 through FIX, a buy from participant 8 through most-exchange's own CLI, and the fill reported to participant 7 at its next Logon:

```
$ deploy/fix send --symbol AAPL --side sell --price 100.00 --qty 10 --clordid S1793473103 --follow
2
logged on as CLIENT7
ExecutionReport ClOrdID=S1793473103 OrderID=20030 ExecID=29 ExecType=NEW OrdStatus=NEW Side=SELL
Symbol=AAPL OrderQty=10 Price=100 CumQty=0 LeavesQty=10 AvgPx=0
Logout MsgSeqNum=20075

$ (cd ../most-exchange/deploy && ./most send --symbol AAPL --side buy --price 100.00 \
      --qty 400000000 --clordid 1793473103 --participant 8 --follow 2)
  NEW  AAPL  orderId=20031 clOrdId=1793473103  leaves 400000000 cum 0 of 400000000
  TRADE  AAPL  orderId=20031 clOrdId=1793473103  last 400000000 @ 100.00 cum 400000000 leaves 0

$ deploy/fix listen --follow 3
logged on as CLIENT7
ExecutionReport ClOrdID=S1793473103 OrderID=20030 ExecID=30 ExecType=TRADE
OrdStatus=PARTIALLY_FILLED Side=SELL Symbol=AAPL OrderQty=10 Price=100 LastQty=4 LastPx=100
CumQty=4 LeavesQty=6 AvgPx=100 PossDup
Logout MsgSeqNum=20078
```

The fill arrived while CLIENT7 was logged out. The adapter wrote its 35=8 to the session offline, and at the Logon the client saw a gap and asked for it: it comes back with PossDup=Y (5.4).

> **`MOST SEND --QTY` IS THE RAW INTEGER**
>
> most-exchange's CLI takes the exchange's fixed-point quantity: `--qty 400000000` is four shares. `--qty 4` would be 0.00000004 of a share, and the FIX client would see `LastQty=0.00000004`.

`deploy/smoke.sh` does all of this, plus market data and a cancel, with new ids each run:

```
$ deploy/smoke.sh
ok: CLIENT7's sell S1793456447 rests
ok: participant 8 crossed it through most-exchange's CLI
ok: CLIENT7 was told of the fill at its next Logon
ok: FEED7 has the security list and AAPL's book
ok: the rest of S1793456447 cancelled
```

It assumes participant 7 has no older order at the same price ahead of its sell in the book.

2.7 Watching the book

```
$ deploy/fix securities --follow 2
logged on as FEED7
SecurityList SecurityReqID=SL-1793472920856 SecurityRequestResult=0 TotNoRelatedSym=2
LastFragment=Y Securities=[AAPL/1/USD/tick=0.01,MSFT/2/USD/tick=0.01]

$ deploy/fix md --symbol AAPL --depth 1 --follow 2
logged on as FEED7
MarketDataSnapshot MDReqID=MD-1793472923401 Symbol=AAPL Entries=[BID 99.98x0.00000089(1)]
```

A subscription prints its snapshot (35=W), then one line per change (35=X) for as long as `--follow` lasts. The quantity above, 0.00000089, is a bid placed through `most` with a raw `--qty 89`.

2.8 Stopping, restarting, crashing

	What happens
`docker compose -f deploy/docker-compose.yml restart fix-7`	SIGTERM: the engine closes, the clean stop is marked, `adapter: stopped`. The next start moves no MsgSeqNum
The JVM killed (`docker exec most-fix-fix-7-1 sh -c 'kill -9 $(pgrep -f AdapterMainKt)'`)	Docker restarts it. For about 10 s the archive refuses its own mark file and the adapter exits; Docker retries. Once started it moves every order-entry session's MsgSeqNum on 10,000 (5.6)

The crash, as it looks:

```
adapter: refused to start: active mark file detected: /data/archive/archive-mark.dat
adapter: refused to start: active mark file detected: /data/archive/archive-mark.dat
order entry: participant 7: the last stop was not clean; MsgSeqNum moved on from 31 to 10031, 10000
Heartbeats (Design.md §8.21)
adapter: started, listening on 0.0.0.0:9876 for 1 order-entry sessions, 0.0.0.0:9877 for 1 market-
data; trading date 20261031, idle backoff
order entry: shard 0 ready
```

2.9 What is development-only

- **The secrets** in `deploy/config/sessions.properties` are committed (their SHA-256 only).
- **The calendar** is UTC and every day of the week, because the exchange's seed opens the shard with the control container's date, on any day (section 4.4). Production uses the exchange's calendar.
- **Shared driver threads and a backing-off idle strategy** (`ADAPTER_OPTS` , `fix.idleStrategy=backoff`): the whole exchange runs on the same machine. Production busy-spins.
- **The fsync is the VM's.** No latency figure taken in Docker Desktop is the adapter's.
- **Dynamic MDC instead of multicast**, as in most-exchange's stack.

2.10 Running without Docker

The e2e scripts run the adapter from its install against most-exchange's installs, and are the reference for a bare-metal layout. `e2e/exchange.sh` writes both files; its node file:

```
fix.bindAddress=localhost
fix.port=29880
fix.logDir=build/e2e/adapter/logs
fix.archiveDir=build/e2e/adapter/archive
fix.aeronDir=build/e2e/adapter/aeron
fix.fileSyncLevel=1
fix.sessionsFile=build/e2e/sessions.properties
fix.storeDir=build/e2e/adapter/store
fix.directoryChannel=aeron:udp?endpoint=localhost:0|control=localhost:40101|control-mode=dynamic
fix.directoryStreamId=100
fix.md.port=29881
fix.md.logDir=build/e2e/adapter/md-logs
fix.md.archiveDir=build/e2e/adapter/md-archive
fix.md.aeronDir=build/e2e/adapter/md-aeron
fix.md.l1.channel=aeron:udp?endpoint=localhost:0|control=localhost:40201|control-mode=dynamic
fix.md.l1.streamId=31
fix.md.l2.channel=aeron:udp?endpoint=localhost:0|control=localhost:40202|control-mode=dynamic
fix.md.l2.streamId=32
fix.md.snapshot.channel=aeron:udp?endpoint=localhost:0|control=localhost:40204|control-mode=dynamic
fix.md.snapshot.streamId=34
fix.calendar.zone=UTC
fix.calendar.rollTime=23:59
fix.calendar.weekdays=MON,TUE,WED,THU,FRI,SAT,SUN
fix.idleStrategy=backoff
```

(the paths shortened from the script's `$RUN`, the channels as it writes them.)

```
ADAPTER_OPTS="-Daeron.threading.mode=SHARED -Daeron.archive.threading.mode=SHARED" \
  adapter/build/install/adapter/bin/adapter build/e2e/adapter.properties
```

The start script carries the JVM flags the adapter needs (4.8); `ADAPTER_OPTS` adds to them.

2.11 Automated checks

Command	What it proves	Needs
`./gradlew build`	Every module's tests, against a real Artio engine and initiator and modelled gateways. What CI runs (`.github/workflows/ci.yml`)	JDK 21
`./e2e/run-e2e.sh`	FIX clients, the adapter and most-exchange's real processes: orders, crosses, offline delivery, market data, a clean restart, a SIGKILL under load and a report after it. Nine steps	most-exchange's installs
`./e2e/run-perf.sh`	The latency baseline (5.8)	The same, and a quiet machine
`deploy/smoke.sh`	The Docker stack, end to end (2.6)	Both stacks up
`./gradlew :spikes:test`	What Artio really does, the assumptions the design rests on. Minutes, with SIGKILLs; after an Artio or Aeron bump	—

2.12 Shutting down

```
docker compose -f deploy/docker-compose.yml stop        # the adapter, cleanly; volumes kept
(cd ../most-exchange/deploy && docker compose stop)
```

`down -v` on the adapter discards its archive and order store. **Never do that alone** while the exchange keeps participant 7’s orders: see 6.7.

3. Production deployment

3.1 The rule that shapes everything

The adapter's disk is its memory. Two things on it carry the delivery promise:

- **order entry's archive**, `fix.archiveDir`, fsynced at `fileSyncLevel` 1. An order goes to the exchange only once the archive has recorded its 35=D, and a report counts as delivered only once the archive has recorded its 35=8 (section 5.3, 5.4);
- **the order store**, `fix.storeDir`, RocksDB with an unsynced write-ahead log, recovered as a prefix after a crash. Everything a host crash can take from it can be rebuilt from the archive.

So the archive's disk must make `fsync` mean what it says, and every order waits for it. On the development laptop the archive's fsync is most of the adapter's latency (Design §8.24, `docs/Measurements.md`). The figures there are a laptop's; measure on the target hardware, with `e2e/run-perf.sh`, before relying on any.

> **DURABILITY IS CLAIMED FOR LINUX**
> On macOS `fsync` does not flush the drive's cache. The design's durability argument holds on Linux with a disk that honours a flush; a laptop proves the ordering, not survival of a power loss.

3.2 Topology

One adapter per participant. Each participant gets its own adapter process, with its order-entry session and a market-data session beside it, on two ports. A participant's sessions are isolated from every other's: a misbehaving client, a slow consumer or a restart touches one participant.

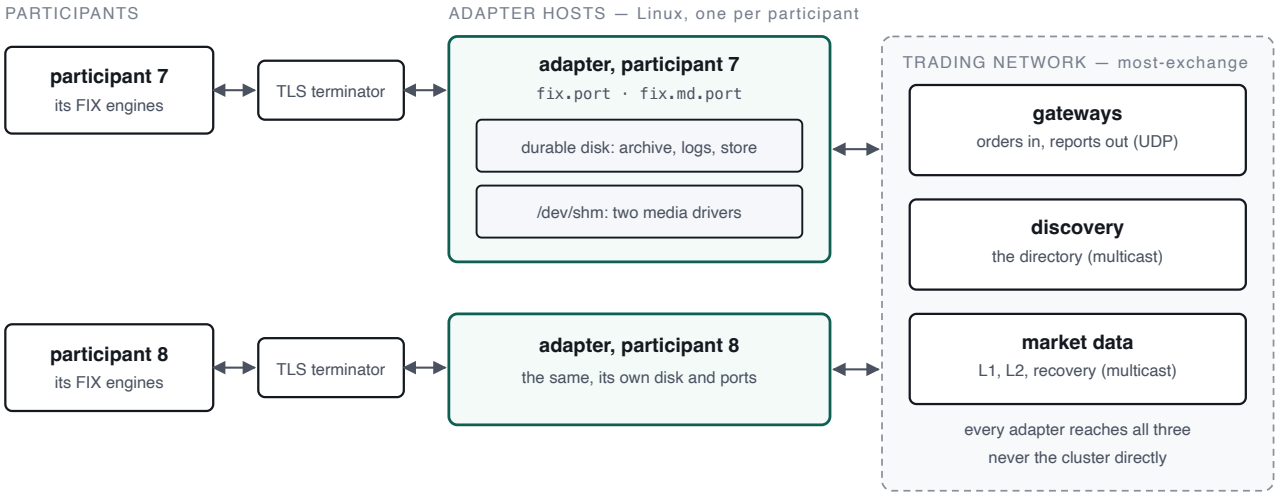


Figure 3.1 One adapter per participant, in production

What each side needs:

Side	Needs
FIX	TCP from the participant to `fix.port` and `fix.md.port`. **Artio does not encrypt:** the password (554) crosses the network as sent. Put a TLS terminator in front, or a private line, never the open internet
Exchange	UDP to the gateways' order entry endpoints, and from their execution report channels; the directory broadcast; the L1, L2 and recovery feeds. In production the directory and the feeds are multicast (most-exchange Operator's Manual, 3.7)
Exchange registry	The participant listed for the gateway the adapter connects through (4.9)

The adapter never connects to the exchange's cluster. It learns the gateways from the directory, and the gateway enforces which participants it may act for.

3.3 Machine preparation

- **Linux**, x86-64. JDK 21 (Temurin is what the project builds and tests on).
- **A disk with power-loss protection** for `fix.archiveDir`, `fix.logDir` and `fix.storeDir`, on one filesystem the adapter owns. They are durable: never on tmpfs, never cleaned at boot.
- `/dev/shm` large enough for two embedded media drivers (`fix.aeronDir`, `fix.md.aeronDir`). An Aeron driver that cannot map a buffer dies of SIGBUS, which reads like a JVM bug (6.5). Size it from the drivers' term lengths; the development stack gives the container 1 GB.
- **Clock synchronisation.** Every FIX message carries SendingTime (52), and clients may reject a stale one. TransactTime (60) on reports is the adapter's clock.
- **Open files**: RocksDB and the archive's segments; raise `LimitNOFILE` above the default.

3.4 Threads and cores

Thread	Busy by default
Order entry's library thread	Yes: `busyspin`
Market data's library thread	Yes: `busyspin`
Each engine's framer and archiver	Artio's own
Each embedded driver's conductor, sender, receiver, archive	Aeron's, `dedicated` threading

`fix.idleStrategy` sets both library threads: `busyspin` (the default), `yielding`, `backoff` or `sleeping`. A busy-spinning thread takes a core whether there is traffic or not; on a shared host, or one with fewer cores than threads, use `backoff` and accept the wake-up latency. Aeron's own threading is set with JVM properties (`-Daeron.threading.mode=SHARED`, `-Daeron.archive.threading.mode=SHARED`), as the development stack does to fit beside the exchange.

3.5 Aeron

- **Two embedded drivers**, one per engine, in `fix.aeronDir` and `fix.md.aeronDir`. Each is deleted and recreated at start. Order entry's carries the gateway links and the directory as well as Artio's own streams: one process, one driver, so a driver fault takes FIX and the exchange links down together.

- **The subscriptions are the adapter's**: `fix.directoryChannel` and the `fix.md.*` feeds are subscriber channels, bound by the adapter's driver on its own host. Use the form the exchange's publisher expects, multicast in production, dynamic MDC (`endpoint=…|control=…`) where multicast is not routed.
- **The gateway's channels come from the directory**, unless listed in the node file (3.6).

3.6 Gateways and failover

By default each shard is reached through the one gateway the directory names. The SDK switches gateways only among those it is given, so a **co-located shard**, a gateway on each cluster node, needs its gateways listed, in the order to try them:

```
fix.shard.0.gateway.0.orderEntryChannel=aeron:udp?endpoint=node-a:20001
fix.shard.0.gateway.0.orderEntryStreamId=20
fix.shard.0.gateway.0.executionReportChannel=aeron:udp?endpoint=0.0.0.0:0|control=node-
a:20002|control-mode=dynamic
fix.shard.0.gateway.0.executionReportStreamId=21
fix.shard.0.gateway.1.orderEntryChannel=aeron:udp?endpoint=node-b:20001
fix.shard.0.gateway.1.orderEntryStreamId=20
fix.shard.0.gateway.1.executionReportChannel=aeron:udp?endpoint=0.0.0.0:0|control=node-
b:20002|control-mode=dynamic
fix.shard.0.gateway.1.executionReportStreamId=21
```

Listed gateways replace the directory's endpoint for that shard. **An endpoint that changes in a later directory is logged, not followed**: the adapter keeps the links it started with until it is restarted. A switch between gateways settles every request in flight by the exchange's resend fence; a request that never reached the exchange is rejected "not sequenced (gateway failover)", and one whose fate the resend cannot tell is restated with an unknown status, never rejected (section 5.5).

3.7 Market data feeds

The adapter subscribes to the same three feeds most-exchange's market data process publishes:

Keys	most-exchange's key	Carries
`fix.md.l1.channel`, `.streamId`	`md.l1.*`	Last trades
`fix.md.l2.channel`, `.streamId`	`md.l2.*`	Depth updates
`fix.md.snapshot.channel`, `.streamId`	`md.snapshot.*`	The recovery feed: an image of each book in turn

A book is trusted once the recovery feed has given an image of it; until then a subscription's snapshot waits. The directory (for the security list) is the order-entry keys' `fix.directoryChannel`.

3.8 Process management

The adapter is one process with a clean stop on SIGTERM: its shutdown hook closes the backends and the engines, waits for the archive to record what was sent, and marks the clean stop in the store (5.6). Give it time, and restart it after a crash, slowly enough for the archive's mark file to age out (~10 s).

An example unit, not run as part of the project's checks:

```
# /etc/systemd/system/most-fix@.service: one instance per participant, most-fix@7
[Unit]
Description=most-exchange FIX adapter, participant %i
After=network-online.target
Wants=network-online.target

[Service]
User=most-fix
Environment="ADAPTER_OPTS=-Xms2g -Xmx2g"
ExecStart=/opt/most-fix/adapter/bin/adapter /etc/most-fix/%i/adapter.properties
KillSignal=SIGTERM
TimeoutStopSec=30
Restart=always
RestartSec=3
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

A restart within ~10 s of a crash fails with `active mark file detected`; `Restart=always` with a few seconds' delay retries until it starts. A clean stop that is cut short (the timeout, a second signal) counts as a crash at the next start.

3.9 Bringing an adapter up

1. **On the exchange:** the participant listed for its gateway (4.9), and the gateway reachable.
2. **The sessions file:** generate each client's secret, store only its SHA-256 (4.7), and hand the secret to the participant out of band with the CompIDs, ports and the Rules of Engagement.
3. **The node file:** the directories on the durable disk, the ports, the exchange's channels, the calendar matching the exchange's (4.4).
4. **Start it** and read the log for `adapter: started`, then `shard N through …` and `shard N ready` for every shard (2.4).
5. **The client logs on.** Its first Logon on a new session is at MsgSeqNum 1, both ways.

Before the first trading day

- The calendar: does `trading date` in the start line name the exchange's trading date?
- A refused Logon (a wrong password) gets `Logon refused` and nothing else; a right one is answered with a Logon (5.2). Try both before handing over.

3.10 Blocking gaps

What production would want that is not built:

> ⚠ **CONTROL API AND METRICS**
>
> `/status`, `/start`, `/stop`, session resets, the trading date override and `/metrics` are designed (Design §9) but not built. Today the adapter is observed through its log alone (5.9), and a session's sequence numbers are reset only by the client (141=Y).

> ⚠ **SESSION SCHEDULE**
>
> A configured session may log on at any time; nothing logs sessions out at the close or resets them at the end of the day (Design §8.2). Sequence numbers run on across days until a client resets.

> ⚠ **TLS**
>
> Artio's acceptor is plain TCP. TLS is terminated in front of the adapter, by something else.

4. Configuration in full

Every key the adapter reads, by the part of the adapter that reads it. Section 7.3 lists them alphabetically. Defaults are the code's; *required* means the adapter refuses to start without it.

4.1 How the adapter is configured

```
adapter /etc/most-fix/7/adapter.properties
```

- **The node file** is the one argument, a Java properties file of `fix.*` keys.
- **Any `fix.*` JVM system property overrides the file's key**, so one file can serve several runs: `ADAPTER_OPTS=-Dfix.idleStrategy=backoff`. With no argument, the system properties alone are the configuration.
- **The sessions file** is named by `fix.sessionsFile` (4.7).
- **Everything is checked before anything opens:** a missing or malformed key, a sessions file that does not parse, a calendar with no trading day. The adapter prints `adapter: refused to start:` and the reason, and exits 1 (6.2).
- **Nothing is re-read while running.** A change takes a restart.

4.2 Order entry's engine

Key	Default	Meaning
`fix.port`	*required*	The order-entry sessions' TCP port
`fix.bindAddress`	`0.0.0.0`	The address it listens on
`fix.sessionsFile`	*required*	The sessions file (4.7), for both engines
`fix.logDir`	*required*	Artio's logs: its session file, sequence and replay indexes, monitoring counters and error log. **Durable**, never deleted at start
`fix.archiveDir`	*required*	Aeron Archive's recordings of every message in and out. **Durable**, never deleted at start
`fix.aeronDir`	Aeron's default directory + `-most-fix`	The embedded media driver's directory, on `/dev/shm`. Deleted and recreated at start
`fix.fileSyncLevel`	`1`	The archive's fsync: `0` none, `1` data, `2` data and metadata. The catalog is synced at the same level. **Below 1 the delivery guarantee does not hold**; a test rig may use 0

> **`FIX.LOGDIR` AND `FIX.ARCHIVEDIR` BELONG TOGETHER**
>
> Artio's indexes in `fix.logDir` describe the recordings in `fix.archiveDir`. Back them up, restore them and move them together, with `fix.storeDir`, or not at all (6.7).

4.3 Order entry's exchange links

Key	Default	Meaning
`fix.storeDir`	*required*	The order store, RocksDB. **Durable**, never deleted at start
`fix.directoryChannel`	*required*	The subscriber channel of the exchange's directory broadcast. Market data's security list reads it too
`fix.directoryStreamId`	*required*	Its stream id
`fix.shard.<id>.gateway.<n>.orderEntryChannel`	—	Optional, per shard: a gateway's order entry channel. n counts from 0, in the order the SDK tries them (3.6)
`fix.shard.<id>.gateway.<n>.orderEntryStreamId`	—	Its stream id
`fix.shard.<id>.gateway.<n>.executionReportChannel`	—	The gateway's execution report channel, subscriber form
`fix.shard.<id>.gateway.<n>.executionReportStreamId`	—	Its stream id

The four `fix.shard.*` keys come as a set per gateway, numbered from 0 without holes; any other key under `fix.shard.` refuses the start. Listed, a shard's gateways replace the one endpoint the directory names for it.

4.4 The trading calendar

The adapter's own calendar gives a **day order** (TimeInForce 0, or absent) its expiry date. It is used for nothing else: a GTD date is the client's and the engine checks it.

Key	Default	Meaning
`fix.calendar.zone`	*required*	An IANA zone, such as `Europe/Amsterdam`
`fix.calendar.rollTime`	*required*	`HH:mm` local to the zone: from this time on, the date is the next trading day
`fix.calendar.weekdays`	`MON,TUE,WED,THU,FRI`	The trading weekdays, by their three-letter names
`fix.calendar.holidaysFile`	none	One ISO date per line (`2026-12-25`), then optional words; # starts a comment

The rule: at local date *d* and time *t*, the trading date is the first trading day on or after *d* if *t* is before the roll, and the first trading day after *d* otherwise. So a Saturday morning names Monday.

Set it to agree with the exchange. Put the roll inside the exchange's closed window, between the close and the next pre-open, when orders are refused anyway. If the two disagree:

- **adapter behind** (D−1 against the engine's D): day orders are rejected `ORDER_EXPIRED`. Visible;
- **adapter ahead** (D+1 against D): a day order is accepted as good till D+1 and **survives the close**. Silent. It happens when the exchange runs a session off its calendar: late, on a weekend, on a holiday. The start line names the adapter's trading date; compare it with the exchange's.

> ⚠ **TRADING DATE OVERRIDE**
>
> `POST /trading-date`, to follow an off-calendar session until the next roll, is designed (Design §9.1) and not built. Today: change the calendar keys and restart.

4.5 Market data

Read only when the sessions file has a market-data session.

Key	Default	Meaning
`fix.md.port`	*required*	The market-data sessions' TCP port. Must not be `fix.port`
`fix.md.bindAddress`	`fix.bindAddress`	The address it listens on
`fix.md.logDir`	*required*	Market data's engine's Artio logs. **Emptied at every start**; must not be `fix.logDir`
`fix.md.archiveDir`	*required*	Its archive. **Emptied at every start**; must not be `fix.archiveDir`
`fix.md.aeronDir`	`fix.aeronDir` + `-md`	Its media driver's directory; must not be `fix.aeronDir`
`fix.md.fileSyncLevel`	`0`	Its archive's fsync. Nothing in it is ever replayed
`fix.md.l1.channel`, `fix.md.l1.streamId`	*required*	The L1 feed (last trades), subscriber form
`fix.md.l2.channel`, `fix.md.l2.streamId`	*required*	The L2 feed (depth updates)
`fix.md.snapshot.channel`, `fix.md.snapshot.streamId`	*required*	The L2 recovery feed

Market data's archive grows within a run, about 200 bytes per 35=W and 35=X sent, and is emptied only at the next start (Design §8.28). Size its disk for the longest run.

4.6 Threads and the trace ring

Key	Default	Meaning
`fix.idleStrategy`	`busyspin`	Both library threads: `busyspin`, `yielding`, `backoff` or `sleeping` (1 ms)
`fix.trace.file`	none: off	Turns the trace ring on, written to this file (5.8)
`fix.trace.ringBytes`	16777216 (16 MiB)	The ring's size: a power of two, at least 512

4.7 The sessions file

One block of `session.<name>.*` keys per FIX session. The name is a label only.

```
session.C7.senderCompId=MOST
session.C7.targetCompId=CLIENT7
session.C7.username=client7
session.C7.passwordSha256=f799b9772b264e618d6311778b7934d7b9db35f687c9fea1ce6eb2fd858fd6b0
session.C7.role=order-entry
session.C7.participantId=7

session.M7.senderCompId=MOST
session.M7.targetCompId=FEED7
session.M7.username=feed7
session.M7.passwordSha256=8ae74e446931569a018a969ab170df419592b1099c5e9476381eeaf8e78b1ea9
session.M7.role=market-data
```

Key	Meaning
`senderCompId`	The adapter's CompID on this session: the client's TargetCompID (56)
`targetCompId`	The client's CompID: its SenderCompID (49)
`username`	The Logon's Username (553)
`passwordSha256`	The SHA-256 of the Logon's Password (554), 64 hex digits
`role`	`order-entry` or `market-data`
`participantId`	The exchange participant, a number. Required for `order-entry`, refused for `market-data`

The adapter refuses to start on a missing or malformed key, two sessions with the same CompIDs, or two order-entry sessions with the same participant.

A Logon is accepted when one configured session matches all of it: its 49 is the session's `targetCompId`, its 56 the `senderCompId`, its 553 the `username`, and the SHA-256 of its 554 the `passwordSha256`, compared in constant time. One session's credentials do not open another. A session logs on at its role's port only; at the other port it is refused like any unknown one.

> **GENERATED SECRETS ONLY**
> The digest is a plain SHA-256, safe for a generated, high-entropy secret, not for a password a person chose. Issue each client a secret, store its digest, and reissue a lost one: it cannot be recovered.

```
openssl rand -base64 24                          # a secret for the client
printf '%s' "$SECRET" | sha256sum | cut -d' ' -f1   # its passwordSha256 (shasum -a 256 on a Mac)
```

4.8 JVM flags

The start script, `bin/adapter`, carries what the adapter needs:

Flag	Why
`--add-opens java.base/jdk.internal.misc=ALL-UNNAMED`	Agrona's unsafe buffers
`--add-opens java.base/sun.nio.ch=ALL-UNNAMED`	Artio's network layer
`-Dfix.codecs.reject_unknown_field=true`	Artio's codecs refuse a tag outside the dictionary. **The acceptor refuses to start without it**, and also if `-Dfix.codecs.no_validation` or `-Dfix.codecs.disable_reject_unknown_enum_value` is set: the adapter's message validation rests on all three

`ADAPTER_OPTS` and `JAVA_OPTS` add to them: the heap, Aeron's threading (`-Daeron.threading.mode=SHARED`, `-Daeron.archive.threading.mode=SHARED` for a small host), and any `-Dfix.*` override.

4.9 What the exchange must agree to

The adapter connects to a gateway, and the gateway enforces which participants it may act for, in the exchange's participant registry (most-exchange Operator's Manual, 4.3). **Every `participantId` in the sessions file must be**

listed for that gateway, or its orders are rejected `UNAUTHORIZED_PARTICIPANT` by the exchange. In the development stack, participant 7 is one of `gw-0`'s.

The feeds' channels and stream ids are those of the exchange's market data process (`md.*`), and the directory's those of its discovery process.

4.10 A complete node file

The development stack's, `deploy/config/adapter.properties`, without its comments:

```
fix.bindAddress=0.0.0.0
fix.port=9876
fix.sessionsFile=/config/sessions.properties
fix.logDir=/data/logs
fix.archiveDir=/data/archive
fix.fileSyncLevel=1
fix.storeDir=/data/store
fix.aeronDir=/dev/shm/fix-aeron

fix.directoryChannel=aeron:udp?endpoint=0.0.0.0:0|control=shard0:40000|control-mode=dynamic
fix.directoryStreamId=100

fix.md.port=9877
fix.md.logDir=/data-md/logs
fix.md.archiveDir=/data-md/archive
fix.md.aeronDir=/dev/shm/fix-aeron-md
fix.md.l1.channel=aeron:udp?endpoint=0.0.0.0:0|control=shard0:40001|control-mode=dynamic
fix.md.l1.streamId=1
fix.md.l2.channel=aeron:udp?endpoint=0.0.0.0:0|control=shard0:40002|control-mode=dynamic
fix.md.l2.streamId=2
fix.md.snapshot.channel=aeron:udp?endpoint=0.0.0.0:0|control=shard0:40004|control-mode=dynamic
fix.md.snapshot.streamId=4

fix.calendar.zone=UTC
fix.calendar.rollTime=23:59
fix.calendar.weekdays=MON,TUE,WED,THU,FRI,SAT,SUN

fix.idleStrategy=backoff
```

5. Daily operations, order flow and monitoring

5.1 The trading day

The exchange's session phases, from pre-open to the close, are the exchange's (most-exchange Operator's Manual, 5.1). The adapter takes part in one way: **it stamps each day order with the trading date its own calendar gives** (4.4). At the calendar's roll the date moves on; day orders from then on carry the next trading day.

- **Day orders** expire at the exchange at the end of their date; the client gets a 35=8 ExecType C, *expired*, as the engine reports it.
- **GTC and GTD orders** stay open across days, at the exchange and in the adapter's store.
- **The store is pruned at start.** A finished request entered before the current trading date, with no report owed and no resend that could bring one, is dropped as the adapter starts; open and ambiguous orders are kept. Nothing is pruned while running, so a long run's store only grows; a restart prunes it.

> ⚠ **SESSION SCHEDULE AND END-OF-DAY RESET**
> The adapter does not log sessions out at the close, nor reset their sequence numbers at the end of the day (Design §8.2, §9.1). Sessions may stay logged on across days; sequence numbers run on until a client logs on with 141=Y.

5.2 Sessions

Logon

A Logon is accepted when it matches a configured session in full (4.7); the adapter answers with its own Logon, HeartBtInt echoed. Anything else gets a Logout, Text `Logon refused`, at MsgSeqNum 1, and the connection is closed. **The Logout never says what was wrong**: CompIDs, username and password are all the same refusal. What the client sees:

```
$ fix listen --port 9876 --sender CLIENT7 --user client7 --password wrong …
Logout MsgSeqNum=1 Text=Logon refused
```

and the adapter's archive shows the attempt, password masked by Artio:

```
8=FIX.4.4|9=093|35=A|49=CLIENT7|56=MOST|34=1|52=20261031-
18:55:20.729|98=0|108=10|141=N|553=client7|554=***|10=252| (AUTH_REJECT)
```

Sequence numbers

Order-entry sessions are persistent. MsgSeqNum is Artio's, across Logouts and adapter restarts. A Logon numbered above what the adapter expects is accepted and the adapter asks for a resend; one below is refused. Numbers reset only when the client logs on with ResetSeqNumFlag (141=Y): the client then gives up every message it had not yet received, and is expected to ask for a mass status (35=AF) to recover its orders.

Market-data sessions are transient: MsgSeqNum 1 both ways at every Logon, and nothing is ever resent. The client logs on with 141=Y, or at 1.

Gaps the adapter asks for

If messages from a client go missing (a gap Artio's own resend has not closed within a second, or a connection that ended with one open), the adapter sends its own ResendRequest, and again each second without progress. After three without progress it logs the session out. The log lines, as formats:

```
order entry: participant <p>: messages <from> to <to> missing; ResendRequest <n> (Design.md §2)
order entry: participant <p>: messages <from> to <to> not received; resend them; logged out after 3
ResendRequests (Design.md §2)
```

The gap is kept across a disconnect and a restart, and asked for again straight after the next Logon.

An id collision blocks a session

The exchange's `clOrdId` is derived from the session's sequence (1.5). If Artio's sequence went back without its `sequenceIndex` rising (its state lost or restored from an old copy), a new request would derive an id an older one holds. The adapter rejects it, logs the session out with Text `session sequence reset required`, and **refuses its Logons until the sequence is reset**. Today the reset is the client's: a Logon with 141=Y lifts the block. The log lines, as formats:

```
participant <p>: id collision, session blocked until its sequence is reset (Design.md §3.3):
<detail>
participant <p>: back at sequenceIndex <i> after a reset; block lifted (Design.md §3.3)
```

5.3 The order path

1. **Artio receives the 35=D** and records it in the archive. The library thread decodes and validates it; a malformed message is a 35=3, an order the exchange cannot carry a 35=8 reject (the Rules of Engagement list each).
2. **The order store gets a row**, *to send*, and the message is consumed.
3. **The archive gate:** the row is sent to the exchange only once the archive has recorded its 35=D, fsynced. Then a host crash cannot leave the exchange holding an order whose 35=D the adapter has forgotten, which a client's PossDup resend would turn into a second order.
4. `newOrder` **to the gateway**, through the SDK. A back-pressured send stays on its row and is tried again in the next duty cycle.
5. **Reports** come back on the gateway's execution report stream, become 35=8 (or 35=9 for a cancel refused), and update the row.

Refused at the adapter, never sent: anything malformed, market and stop orders, IOC and FOK, a replace (35=G), and every order and cancel while the exchange is unavailable: before the directory, before a shard's opening mass status, with no gateway. Those get Text `exchange unavailable`, and the client may retry with a new ClOrdID.

A cancel sent before its order's first report is **parked** on its row and sent when the report brings the exchange's order id; if that report is terminal, the cancel is answered *too late*.

5.4 Delivery

A report is consumed from the exchange only once its 35=8 is fsynced in the archive. The store keeps, per participant and shard, the first report sequence number not yet durably delivered; it advances only past reports whose 35=8 the archive has recorded. After any restart the adapter asks the exchange to resend from there.

- **A report for a logged-out client** is written to its session offline, numbered by Artio and archived. At the next Logon the client sees the gap and asks for a resend; the report comes back PossDup=Y (2.6). Delivery does not depend on how long the exchange keeps its reports.
- **A report delivered twice** across a restart carries the same ExecID, the exchange's report sequence number, so the client can drop the second.
- **A 35=8 Artio refuses** (back-pressure) is not consumed: the report waits on the exchange's stream until Artio takes it.

5.5 An order whose fate is unknown

When a gateway fails over, the exchange's resend fence settles every request in flight. A request that never reached the exchange is rejected `not sequenced (gateway failover)`, and the client may send it again with a new ClOrdID. One the fence cannot settle, because the resend was truncated, is **ambiguous**: not open at the exchange, but whether it filled is unknown.

The adapter **never reports an ambiguous order as rejected, cancelled, expired or done**: that would invite the client to re-enter a quantity that may have traded. It restates the order as the client last knew it, ExecType I, with OrdStatusUnknown (5001) = Y and Text `STATUS UNKNOWN: not open at the exchange; fills since CumQty=… unrecoverable until reconciled`. A cancel for it is answered locally with the same flag. A later report for it clears the flag. The log line, as a format:

```
participant <p>: order <clOrdId> open in the store, not stated by the mass status: ambiguous
(Design.md §3.4)
```

Reconciling an ambiguous order's fills is manual until the exchange has a drop copy: the exchange's operator can see the trades.

5.6 Restarts and crashes

A clean stop

SIGTERM (Ctrl-C, `docker compose stop`, `systemctl stop`) runs the shutdown hook: the library threads stop, the engines close (Artio waits for the archive to record everything it sent), and only then is the clean stop marked in the store. `adapter: stopped` is the last line.

At start

Before the ports bind, the adapter:

1. loads the store, pruning what retention allows (5.1);
2. refuses to start if Artio's state looks wiped while the store holds the participant's state, or Artio's `sequenceIndex` is below the stored one (6.2);
3. rebuilds any request the archive has and the store lost (a host crash), from Artio's archive, one session at a time;
4. **after an unclean stop**, moves every order-entry session's MsgSeqNum on 10,000, by writing that many Heartbeats offline. A 35=8 can have reached the client and not the archive; without the jump the next Logon would reuse its number and the client would end the session. The client sees one gap fill at its next Logon and carries on;
5. hands the requests in flight back to the SDK, whose resend and mass status settle them.

```
order entry: participant 7: the last stop was not clean; MsgSeqNum moved on from 31 to 10031, 10000
Heartbeats (Design.md §8.21)
```

and, when requests were in flight at the crash (from the e2e's step 8):

```
order entry: shard 0: 2 requests restored in flight
```

After a crash

- **~10 s of** `active mark file detected` : the archive's mark file is still live. Retry; a supervisor does (3.8).
- **Then everything is recovered**: orders, reports owed, offline reports, and the gap fill. The e2e's step 8 kills the adapter under load from two clients and finds every order answered and none unknown.
- **A report the client had already received** before the crash may come again under a new MsgSeqNum, with the same ExecID.

5.7 Market data operations

- **A subscription** is a 35=V; the client gets a snapshot (35=W) per instrument, then incremental updates (35=X). It lives in memory and ends with the session.
- **A book the adapter does not trust yet** (it has not seen its recovery image, or lost sequence) sends nothing: the snapshot waits, updates stop, and a fresh 35=W follows once it is trusted again.
- **A client that cannot keep up** (Artio refuses a 35=X) gets a fresh 35=W once it can, never a wrong book; it is not logged out.
- **A market-data restart** of the exchange's own process is seen as a resynchronisation; the adapter needs nothing done.
- **Market data's archive grows within a run** (4.5); the next start empties it.

5.8 Measuring

The adapter has its own measuring sender and report, built for one rule: report both service time (from the send) and response time (from the scheduled send), so a stalled sender cannot hide queueing (Design §10, the `perf-claim` skill).

```
./e2e/run-perf.sh        # the baseline beside `most load`, against most-exchange's installs
fix perf --symbol AAPL --mid 100.00 --rate 1000 --count 60000 --log run.log …
fix report --log run.log --out report/      # report.html, .hgrm, CSV
```

The trace ring records each order's stages inside the adapter (in, archive gate, sent, report, out, durable) when `fix.trace.file` is set; `fix report --trace FILE` joins it to the sender's log by MsgSeqNum and cuts the service time at each stage. Its times are the host's monotonic clock, so sender and adapter must share the host.

Figures taken in Docker Desktop or on a laptop are not production's (3.1). Every measurement taken so far is in `docs/Measurements.md` , with its conditions.

5.9 Monitoring

Until `/metrics` and `/status` exist, **the log is the monitoring surface**. Lines worth alerting on:

Line	Means
`adapter: refused to start:`	A start failed; the reason follows (6.2)
`the last stop was not clean`	A crash, recovered; find out why it crashed
`the library lost the engine; reports wait until it is back`	Artio's library and engine lost each other; reports are held
`id collision, session blocked`	A session needs its sequence reset (5.2)
`… ambiguous`	An order's fate is unknown; reconcile it (5.5)
`logged out after 3 ResendRequests`	A client is not resending what it owes
`report for clOrdId …, which the store does not hold; consumed`	The exchange has an order the store lacks (6.4)
`shard N refused participant P's request`	The exchange's registry does not grant the participant (4.9)
`order-entry:` or `market-data:` followed by a stack trace	An unexpected error on a library thread

> ⚠ **METRICS AND STATUS**
>
> Counters (messages, rejects, fates, back-pressure, gateway switches), gauges (sessions, open orders, durable lag) and internal latencies are designed for `/metrics` (Design §9.2), not built.

6. Troubleshooting and maintenance

6.1 First moves

1. **Is it running, and did it start?** The log's first lines (2.4): `adapter: started`, then `shard N through …` and `shard N ready` for each shard. A start that failed says `adapter: refused to start:` and why (6.2).
2. **Is the exchange there?** No `shard N through` means no directory; no `shard N ready` means the gateway's opening mass status has not finished. Both are on the exchange's side of the link.
3. **Is it one client or all?** One client: its Logon, its sequence numbers, its messages (6.3, 6.4; the Rules of Engagement's troubleshooting chapter is the client's half). All clients: the adapter or the exchange.
4. **What did the wire carry?** The archive has every FIX message in and out (6.8).

6.2 Startup failures

Every one of these happens before the ports bind, so no client sees a half-started adapter.

`adapter: refused to start: …`	**Cause and fix**
`missing required configuration key: <key>`	Add the key (section 4)
`<key> must be a port number, … must be 0, 1 or 2, … must be an Aeron stream id, a number`	A malformed value
`fix.md.port must not be fix.port: market data has its own engine`	Give market data its own port
`fix.md.logDir must not be order entry's directory: market data's is emptied at every start` (also `archiveDir`, `aeronDir`)	Market data's directories are wiped at start: they must be their own
`<file>: sessions [A, B] share CompIDs … / … share participantId …`	Two sessions with one identity, or two order-entry sessions for one participant (4.7)
`<file>: session.<n>.passwordSha256 must be 64 hex digits …`	Store the digest, not the secret
`fix.calendar.zone: not a time zone`, `fix.calendar.rollTime must be HH:mm`, `fix.calendar.weekdays: <x> is not one of …`	The calendar (4.4)
`Artio accepts unknown tags: start the JVM with -Dfix.codecs.reject_unknown_field=true`	The JVM was started without the start script's flags (4.8)
`active mark file detected: …/archive-mark.dat`	Within ~10 s of a crash. Wait and start again; a supervisor retries (3.8)
`session <s>: Artio does not know it, yet the order store holds state for participant <p>. Artio's state was lost, …`	`fix.logDir`/`fix.archiveDir` were wiped or replaced while `fix.storeDir` was kept. Restore them together, or see 6.7
`participant <p>: Artio's sequenceIndex <i> is below the stored <j>. …`	Artio's state is older than the store's: an old backup restored. Restore all three directories from one moment

`adapter: refused to start: …`	**Cause and fix**
`order entry: … refusing to start rather than rebuild it for the wrong participant`	The archive replayed a message whose SenderCompID is not its session's client. The archive and the sessions file disagree: CompIDs changed under an existing session
`order entry: the offline sessions were not handed over, their archives replayed, within 10 s`	The startup replay did not finish; usually a starved host. Look for other errors above it

6.3 Logon problems

Symptom	Cause
`Logout … Text=Logon refused`, at MsgSeqNum 1	No configured session matches the Logon's 49, 56, 553 and the SHA-256 of 554 all at once, or the session is at the other role's port. The Logout never says which: check each, starting with the port
Logged out, MsgSeqNum lower than expected	The client logged on below what the adapter expects. The client's sequence store and the adapter's disagree: the client's was reset, or the adapter's state is older than the client's. The client may log on with 141=Y, giving up what it had not received
Logged out at once, `session sequence reset required`	The session is blocked by an id collision (5.2). A Logon with 141=Y lifts it
Logged out, `messages <a> to <b> not received; resend them`	The client did not answer the adapter's ResendRequests (5.2). It is asked again after its next Logon
The connection is refused	The adapter is not listening: not started, still in the ~10 s after a crash, or `fix.bindAddress` is not reachable from the client

6.4 Order flow problems

Symptom	Cause
Every order rejected `exchange unavailable`	No complete directory yet, a shard's opening mass status not finished, or no gateway. Check 6.1 step 2
Rejected `participant not served`	The exchange's gateway session does not hold the participant
Rejected by the exchange, `UNAUTHORIZED_PARTICIPANT`	The participant is not granted to the adapter's gateway in the exchange's registry (4.9)
Rejected `ORDER_EXPIRED` on every day order	The adapter's calendar is behind the exchange's trading date (4.4)
A day order survived the close	The adapter's calendar is ahead: the exchange ran a session off its calendar (4.4)
Rejected `not sequenced (gateway failover)`	A gateway switch; the order never reached the exchange. Safe to resend with a new ClOrdID
An order restated with OrdStatusUnknown=Y	Its fate is unknown (5.5)
`report for clOrdId <id>, which the store does not hold; consumed`	The exchange reported an order the store lacks. After a host crash, for a request the archive did not have either; or the adapter's state was wiped while the exchange kept the participant's orders (6.7)
A fill the client never saw	The report is in the archive if it was delivered (6.8); a client that logs on with 141=Y gives up every report it had not received, and recovers with 35=AF

6.5 Aeron and Docker problems

Symptom	Cause
A JVM fatal error (SIGBUS or SIGSEGV) in an Agrona or Aeron frame, `AtomicCounter`, `UnsafeBuffer`	A memory-mapped Aeron file could not be backed: `/dev/shm` (or a tmpfs volume) full. Size it (3.3)
The same in **several containers at once**	Docker Desktop's VM ran out of memory: tmpfs and `/dev/shm` are the VM's RAM. Seen when a large Gradle build ran in another container beside both stacks. Give the VM more memory, or run heavy jobs outside it
`Restarting` over and over	Read `docker logs most-fix-fix-7-1`: a configuration error repeats on every start
Healthy, but no `shard 0 ready`	No directory: most-exchange's discovery is not up, or the channel's control address is not `shard0:40000`
`channel error - Cannot assign requested address`	A subscriber channel names an address the adapter's driver does not own. Subscriptions use `endpoint=0.0.0.0:0` with the publisher's control address

6.6 After a crash

1. **Let it restart.** ~10 s of `active mark file detected`, then a normal start with `the last stop was not clean; MsgSeqNum moved on …`.
2. **Find out why it crashed**: the JVM's `hs_err_pid*.log`, the kernel log for the OOM killer, Artio's error log (6.8).
3. **Clients** see one gap fill at their next Logon and carry on (5.6). Reports owed while the adapter was down are delivered: the store resumes each participant from its durable report sequence.

6.7 Maintenance procedures

Adding or removing a session

Edit the sessions file and restart. A new order-entry session needs its participant granted on the exchange (4.9). Removing a session whose participant still has open orders leaves them open at the exchange; cancel them first.

Rotating a client's secret

Generate a new secret (4.7), replace the session's `passwordSha256`, restart. The session's sequence numbers and orders are untouched; the client logs on with the new secret.

Backing up and restoring

The adapter's state is three directories that must stay consistent with each other: `fix.logDir`, `fix.archiveDir` and `fix.storeDir`. Copy them together, with the adapter stopped, and restore them together. A store newer than Artio's state, or older, refuses the start (6.2), which is the guard doing its job: derived order ids would repeat.

Starting a participant afresh

> **NEVER WIPE THE ADAPTER'S STATE ALONE**
>
> While the exchange still holds the participant's orders, a fresh adapter starts at `sequenceIndex` 0 again: its new orders' exchange ids repeat ones the exchange already has, and reports for the old orders reach a store that does not know them. Cancel the participant's orders at the exchange first (most-exchange's `most cancel`, or the exchange's operator), then stop the adapter and remove all three directories. In Docker: `docker compose -f deploy/docker-compose.yml down -v`, and delete the client's own sequence store.

Resetting a session's sequence numbers

Today, only the client can: a Logon with ResetSeqNumFlag (141=Y). The client gives up every report it had not received and asks for a mass status (35=AF).

> **⚠ OPERATOR RESET**
>
> `POST /sessions/{id}/reset` and the end-of-day `POST /reset` are designed (Design §9.1), not built.

Upgrading the adapter

Stop it cleanly, replace the install, start it. The store's rows are versioned, and a row of an unknown version refuses the start rather than being misread. Artio and Aeron move only together with most-exchange's Aeron.

6.8 Reading the logs

- **The adapter's own log** is stderr: one line per event, most naming the Design section they follow from.
- **Artio's error log and counters** are in `fix.logDir` (and `fix.md.logDir`), not the system's temp directory.
- **The archive** holds every FIX message each way, and Artio's `FixArchivePrinter` prints it from the running adapter's driver. In the Docker stack:

```
docker exec most-fix-fix-7-1 sh -c 'java \
  --add-opens java.base/jdk.internal.misc=ALL-UNNAMED --add-opens java.base/sun.nio.ch=ALL-UNNAMED
\
  -cp "/opt/fix/adapter/lib/*" uk.co.real_logic.artio.engine.logger.FixArchivePrinter \
  --aeron-dir-name=/dev/shm/fix-aeron --aeron-channel=aeron:ipc \
  --fix-dictionary=nl.lamia.most.exchange.fixadapter.codecs.FixDictionaryImpl \
  --log-file-dir=/data/logs --delimiter="|" --query-stream-id=1'
```

`--query-stream-id=1` prints what clients sent, `2` what the adapter sent. Each line ends with Artio's verdict on it, `(OK)` or, for a refused Logon, `(AUTH_REJECT)`. Passwords are masked. One line from the development stack, the adapter's report of a cancel:

```
8=FIX.4.4|9=198|35=8|49=MOST|56=CLIENT7|34=20068|52=20261031-
18:55:15.778|37=20029|11=RC1793472897|41=RB1793472897|17=28|150=4|39=4|55=AAPL|48=1|22=8|54=1|38=25
|44=99.5|59=0|151=0|14=0|6=0|60=20261031-18:55:15.779|10=125|
```

11 is the cancel's ClOrdID, 41 the order's, ExecType (150) 4 and OrdStatus (39) 4, cancelled.

6.9 Consolidated list of what is not implemented

Designed, not built	Where
Control API: `/status`, `/start`, `/stop`, session and end-of-day resets, trading date override	Design §9.1
`/metrics`: counters, gauges, internal latencies	Design §9.2
Session schedule: logon windows, logout at the close, reset at end of day	Design §8.2
Several order-entry sessions for one participant	Design §8.27
Client fields carried to the exchange (Account and others)	Design §5.5
An order-entry throttle	Design §8.25
TLS on the acceptor	Not planned: terminated in front
Market by order (L3) market data	Waits on the exchange

7. Index

References are to section numbers. **Bold** marks where a term is defined or explained.

7.1 Terms and concepts

A–C

Ambiguous order, see Unknown status
Archive **1.2**, 3.1, 4.2, 5.4, 6.8
Archive gate **5.3**, 3.1, 5.8
AUTH_REJECT 5.2, 6.8
Backup and restore **6.7**, 4.2
Busy-spin **3.4**, 4.6
Calendar, trading **4.4**, 2.9, 5.1, 6.4
Cancel, parked **5.3**
Clean stop **5.6**, 2.8, 3.8
clOrdId, derived **1.5**, 5.2, 6.7
CompID 1.6, **4.7**, 6.3
Crash 2.8, **5.6**, 6.6
Crash skip, see MsgSeqNum

D–G

Day order 4.4, **5.1**, 6.4
Delivery, guaranteed 1.1, **5.4**
Development stack **2.3**, 2.9
Directory 1.3, 2.4, **4.3**, 6.1
Docker **2.3**, 2.8, 6.5
Dynamic MDC 2.3, 3.5
Embedded media driver **1.2**, 3.5, 4.2
ExecID **1.5**, 5.4
Exchange unavailable 5.3, **6.4**
fileSyncLevel 3.1, **4.2**, 4.5
FixArchivePrinter **6.8**
Gap, asked for by the adapter **5.2**, 6.3
Gateway failover **3.6**, 5.5, 6.4
GTC, GTD orders 5.1

H–O

Health check 2.3
Holidays file **4.4**
Id collision **5.2**, 6.3
Idle strategy 3.4, **4.6**
JVM flags **4.8**, 6.2
Logon, refused **5.2**, 6.3
Mark file, archive 2.8, 3.8, 6.2, **6.6**
Market data 1.2, 3.7, **4.5**, 5.7
Mass status (35=AF) 5.2, 6.4
MsgSeqNum **5.2**, 6.3
MsgSeqNum, moved on after a crash 2.8, **5.6**, 6.6
Node file 1.4, **4.1**, 4.10
Offline session 2.6, **5.4**
Order store **1.2**, 3.1, 4.3, 5.1, 6.7
OrdStatusUnknown (5001) **5.5**, 6.4

P–S

Participant **1.1**, 1.6, 4.9
Password digest **4.7**, 6.7
PossDup 2.6, 5.4
Prices and quantities, fixed point **1.5**, 2.6
Pruning the store **5.1**
Registry, the exchange's **4.9**, 6.4
reportSeq **5.4**, 6.6
ResetSeqNumFlag (141) 5.2, 6.3, 6.7
Restart policy 2.3, 3.8
Role **1.1**, 4.7
sequenceIndex 1.5, 5.2, 6.2, 6.7
Sessions file 1.4, **4.7**
SIGBUS 3.3, **6.5**
Smoke test 2.6, 2.11
systemd **3.8**

T–Z

TLS 1.3, 3.2, 3.10
Topology **3.2**
Trace ring 4.6, **5.8**
Trading date 2.4, **4.4**, 5.1
Transient session 1.6, 5.2
UNAUTHORIZED_PARTICIPANT 4.9, 6.4
Unknown status **5.5**, 3.6, 6.4
Wipe of the adapter's state 2.12, **6.7**

7.2 Commands

The adapter

Command	Section
`adapter <node file>`	2.10, 4.1
`docker compose -f deploy/docker-compose.yml up -d --build`	2.3
`docker compose -f deploy/docker-compose.yml restart fix-7`	2.8
`docker compose -f deploy/docker-compose.yml stop` / `down -v`	2.12, 6.7
`docker logs most-fix-fix-7-1`	2.4, 6.5
`FixArchivePrinter`	6.8

Clients and tools

Command	Section
`deploy/fix send`, `cancel`, `status`, `mass-status`, `listen`	2.5, 2.6
`deploy/fix securities`, `md`	2.5, 2.7
`deploy/smoke.sh`	2.6, 2.11
`fix perf`, `fix report`, `fix report --trace`	5.8
`most send` (most-exchange)	2.6

Build and test

Command	Section
`./gradlew build`	2.2, 2.11
`./gradlew :adapter:installDist :tools:installDist`	2.2
`./e2e/run-e2e.sh`	2.11
`./e2e/run-perf.sh`	2.11, 5.8
`./gradlew :spikes:test`	2.11

7.3 Configuration keys and environment variables

The node file

Key	Section
`fix.aeronDir`	4.2
`fix.archiveDir`	4.2
`fix.bindAddress`	4.2
`fix.calendar.holidaysFile`	4.4
`fix.calendar.rollTime`	4.4

Key	Section
`fix.calendar.weekdays`	4.4
`fix.calendar.zone`	4.4
`fix.directoryChannel`	4.3
`fix.directoryStreamId`	4.3
`fix.fileSyncLevel`	4.2
`fix.idleStrategy`	4.6
`fix.logDir`	4.2
`fix.md.aeronDir`	4.5
`fix.md.archiveDir`	4.5
`fix.md.bindAddress`	4.5
`fix.md.fileSyncLevel`	4.5
`fix.md.l1.channel`, `fix.md.l1.streamId`	4.5
`fix.md.l2.channel`, `fix.md.l2.streamId`	4.5
`fix.md.logDir`	4.5
`fix.md.port`	4.5
`fix.md.snapshot.channel`, `fix.md.snapshot.streamId`	4.5
`fix.port`	4.2
`fix.sessionsFile`	4.2, 4.7
`fix.shard.<id>.gateway.<n>.executionReportChannel`	4.3, 3.6
`fix.shard.<id>.gateway.<n>.executionReportStreamId`	4.3, 3.6
`fix.shard.<id>.gateway.<n>.orderEntryChannel`	4.3, 3.6
`fix.shard.<id>.gateway.<n>.orderEntryStreamId`	4.3, 3.6
`fix.storeDir`	4.3
`fix.trace.file`	4.6
`fix.trace.ringBytes`	4.6

The sessions file

Key	Section
`session.<name>.participantId`	4.7
`session.<name>.passwordSha256`	4.7
`session.<name>.role`	4.7
`session.<name>.senderCompId`	4.7
`session.<name>.targetCompId`	4.7
`session.<name>.username`	4.7

JVM and environment

Name	Section
`ADAPTER_OPTS`, `JAVA_OPTS`	4.8
`-Daeron.threading.mode`, `-Daeron.archive.threading.mode`	3.4, 4.8
`-Dfix.codecs.reject_unknown_field=true`	4.8, 6.2
`-Dfix.<key>`, overriding the node file	4.1

7.4 Further reading

Document	For
Rules of Engagement (`docs/RulesOfEngagement.pdf`)	The participant's side: sessions, messages, tags, order types
`docs/Design.md`	The authoritative specification; §8 is the open list
`docs/Rationale.md`	Why each rule exists
`docs/Measurements.md`	Every latency figure taken, with its conditions
`deploy/README.md`	The Docker setup
most-exchange's `docs/OperatorManual.pdf`	The exchange: shards, gateways, the participant registry, the trading day
most-exchange's `docs/Adapters.md`	The contract this adapter implements